



Domain Specific Language Design (2IMP20): Syntax



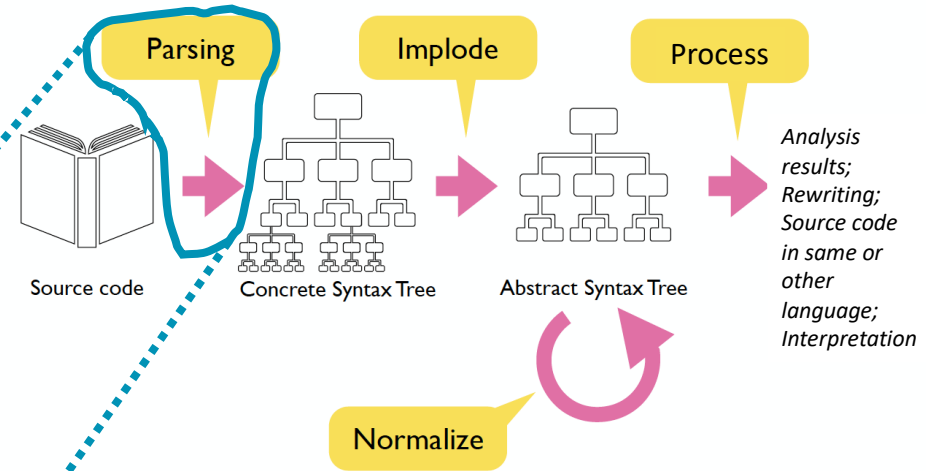
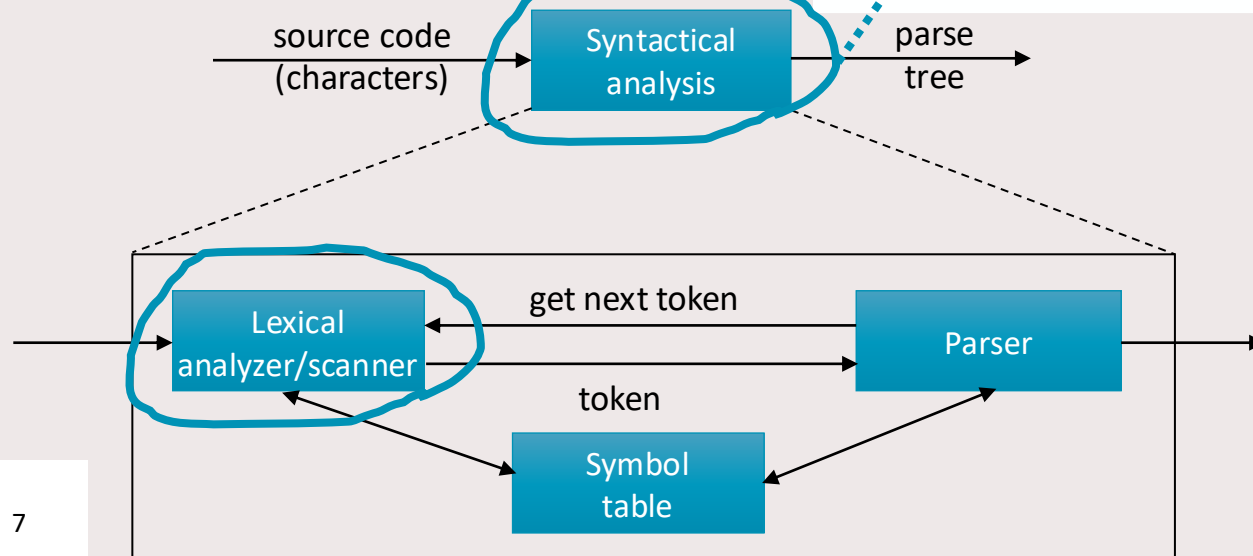
Loek Cleophas



Ivan Kurtev

Lexical analysis a.k.a. scanning

- Tasks and organization of *lexical analyzer* a.k.a. *scanner*
- Specification of lexical *tokens* via regular expressions



Lexical syntax

Tasks of the lexical analyzer:

- reading the input and producing of tokens
 - each token *represents one logical piece of the source* – keyword, name of a variable, value, etc. (*corresponds to a syntactic category*)
 - each token *may have optional attributes*
 - including e.g. a *lexeme* (e.g. “myVar”, “+”)
- elimination of layout and comments?
- keeping track of position information? → line and column attributes

Lexical syntax

What are relevant tokens?

```
for (int k = 0; k < myArray[5]; ++k) {  
    cout << k << endl;  
}
```

for	{
int	}
cout	;
<<	<
=	[
(	]
)	++

Identifier

IntegerConstant

Lexical syntax

Scanning is not trivial, especially in “old” languages

- FORTRAN: whitespace is irrelevant

```
DO I = 1, 25
```

```
DOI = 1, 25
```

```
DOI = 1.25
```

- First and second line represent a loop running from $i = 1$ to 25
- Third line is an assignment of the value 1.25 to the variable DOI

→ Difficult to tell how to partition the input

Lexical syntax

Tokens *categorize lexemes* by what information they represent
(→ notion of *syntactic category*)

Some tokens might be associated with only a single lexeme:

- Tokens for keywords like **if** and **while** probably only match those lexemes exactly

Some tokens might be associated with lots of different lexemes:

- All variables, all possible numbers, all possible strings, etc.

Lexical syntax – How to define?

Regular expressions are a family of descriptions that can be used to capture a class of languages (*regular languages*)

- Provide a compact and human-readable description of the language
- Often used as basis for compiler/language tools, e.g. ANTLR,
(f) lex

Lexical syntax

A **regular expression (RE)** r over an alphabet Σ corresponds to language $L(r)$

1. ϵ is a RE and corresponds to $\{\epsilon\}$
2. $a \in \Sigma$ is a RE and corresponds to $\{a\}$
3. Suppose r and s are REs corresponding to the languages $L(r)$ and $L(s)$
 - a. **alternative** $(r) \mid (s)$ is a RE $\Leftrightarrow L(r) \cup L(s)$
 - b. **concatenation** $(r)(s)$ is a RE $\Leftrightarrow L(r) L(s)$
 - c. **Kleene closure** $(r)^*$ is a RE $\Leftrightarrow (L(r))^*$
 - d. **plus** $(r)^+$ is a RE $\Leftrightarrow (L(r))^+$
 - e. **optional** $(r)?$ is a RE $\Leftrightarrow L(r) \cup \{\epsilon\}$
 - f. **brackets** (r) is a RE $\Leftrightarrow L(r)$

Operators are left-associative and priorities are $? > \{*, +\} > \text{concatenation} > \mid$

Lexical syntax

Extension of regular expressions:

1. $a \mid b \mid c \mid d \mid \dots \mid z \in \Sigma$ is a RE and corresponds to $\{a, b, c, d, \dots, z\}$, then this is abbreviated as $[a-z]$ (a so-called *character class*); also e.g. $[+\backslash-]$

2. A *regular definition* over alphabet Σ has the form:

$$id_1 ::= re_1$$
$$id_2 ::= re_2$$
$$\dots$$
$$id_n ::= re_n$$

where id_i are different names and each re_i is a RE over alphabet

$$\Sigma \cup \{id_1, id_2, \dots, id_{i-1}\}$$

Thus, in re_i only names occur which are already defined

Lexical syntax

Floating point numbers specified in Rascal

```
lexical UnsignedInt = [0] | ([1-9][0-9]*);  
  
lexical SignedInt = [+|-]? UnsignedInt;  
  
lexical UnsignedReal = UnsignedInt [.] [0-9]+ ([eE] SignedInt)?;  
lexical UnsignedReal = UnsignedInt [eE] SignedInt;  
  
lexical Number = UnsignedInt | UnsignedReal;
```

0 1 14 0.1 3e4 3.014e-7

00 01 04.1 3e04 3.14e-07

Lexical syntax – Challenges in scanning

- How do we determine which lexemes are associated with which token?
- When there are *multiple ways* we could scan the input, how do we know which one to pick?

- → Lexical ambiguities

T_For	for
T_Identifier	[A-Za-z_][A-Za-z0-9_]*

- How to classify `fort` ?

Lexical syntax

Lexical ambiguities: *conflict resolution*

- Assume all tokens are specified as *regular expressions*.
- Algorithm: Left-to-right scan.
- Always *match the longest possible prefix* of the remaining text.
- When two regular expressions apply, *choose the one with higher “priority”*.
 - Simple priority system: pick the rule that was defined first (used by `(f) lex`).

Lexical syntax

Resolution of ambiguities

- Longest match is preferred
- If two alternatives recognize the same sequence of characters, the alternative occurring first in the specification is chosen

BEGIN	[<i>sym</i> := <i>beginsym</i>]
IF	[<i>sym</i> := <i>ifsym</i>]
...	
<i>letter</i> (<i>letter</i> <i>digit</i>)*	[<i>sym</i> := <i>idsym</i>]
<i>digit</i> (<i>digit</i>)*	[<i>sym</i> := <i>intrepsym</i>]
:=	[<i>sym</i> := <i>becomessym</i>]

Lexical syntax

Rascal offers the class: `lexical` **and** `keyword`

Repeat zero (*) or one (+) or more times

```
lexical Id = ([a-z0-9] !<< [a-z][a-z0-9]* !>> [a-z0-9]) \ Keywords;  
keyword Keywords = "if" | "then" | "else" | "fi"
```

A **character class**: `Id`
starts with a lowercase letter

Lexical syntax

Rascal offers the following possibilities for lexical disambiguation:

- longest match
- prefer keywords

```
lexical Id = ([a-z0-9] !<< [a-z][a-z0-9]* !>> [a-z0-9]) \ Keywords;  
keyword Keywords = "if" | "then" | "else" | "fi"
```

A **lexical restriction**: is `aaa` three, two or one identifier?
`!>>` can be used to define **longest match**. `!<<` prohibits that an identifier is preceded by characters.

Lexical syntax — Complications

- *White space and comments* are serious challenges when defining the lexical syntax
- Rascal *inserts layout* between the elements in the right hand side of a production rule (see next slides)
- In some languages white space has semantics, e.g. Python, Haskell, COBOL, etc.
 - How?
- Comments may be *context-free* instead of regular, for instance, *nested comments*
 - Give a language that supports nested comments

Lexical syntax

Rascal offers a special lexical class: `layout`

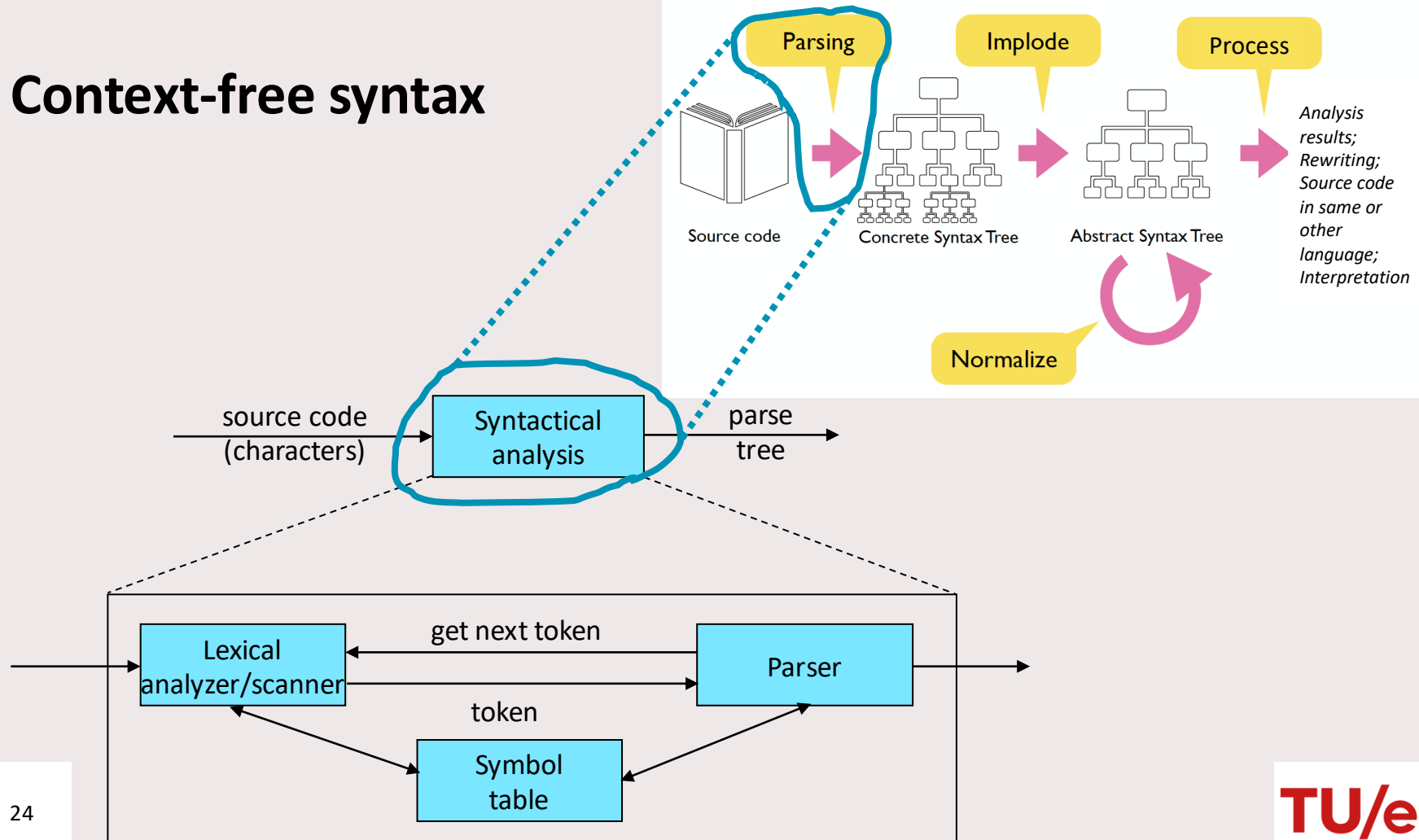
```
layout Layout = WhitespaceAndComment* !>> [\ \t\n\r%];  
lexical WhitespaceAndComment =  
    [\ \t\n\r]  
    | "\"" ![%]+ "\""   
    | "%%" ![%]* $ ;
```

"\$" represents "end of line"

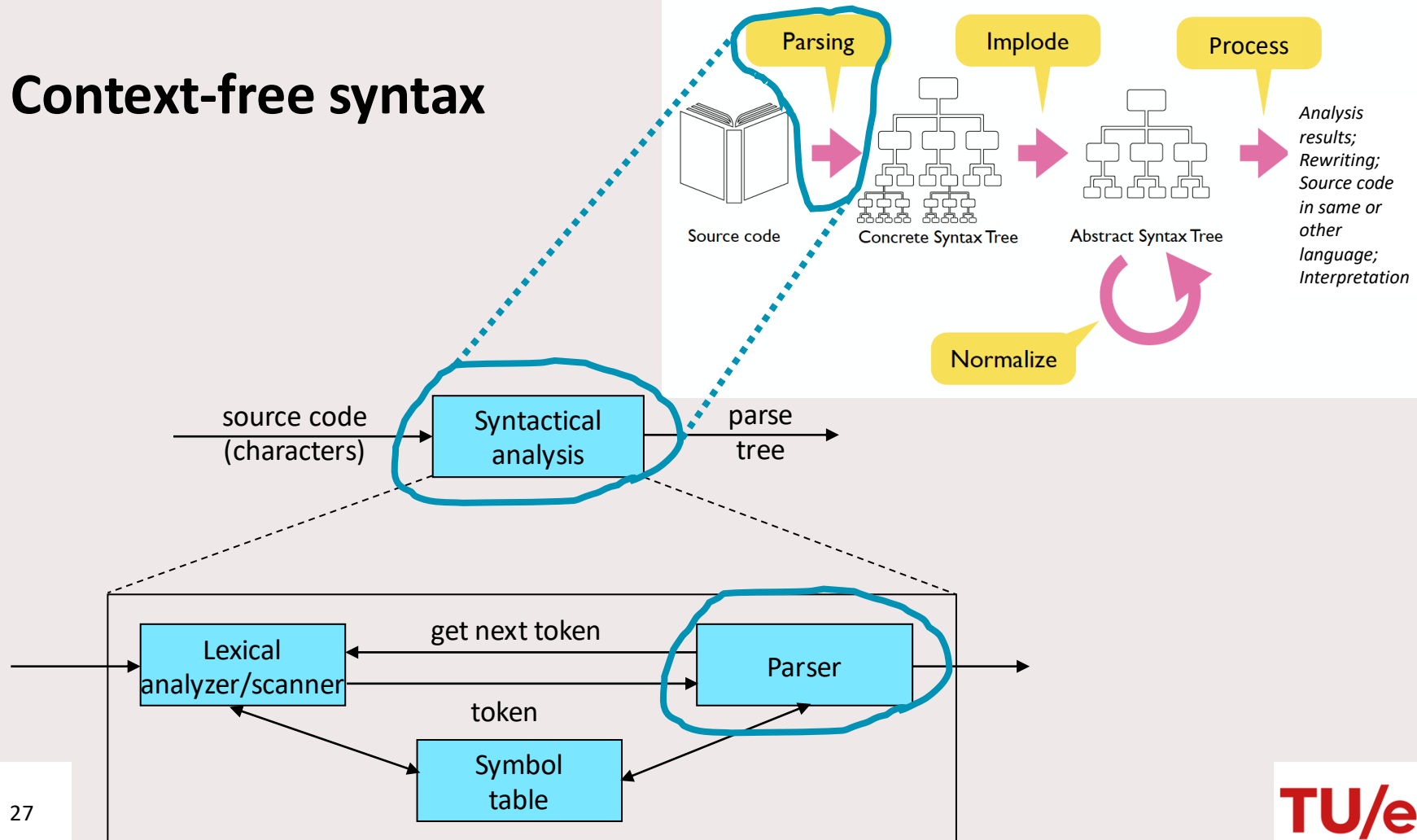
Questions?



Context-free syntax



Context-free syntax



Context-free syntax

Reading material:

- Chapter 2 (“Syntax analysis”) of Introduction to Compiler Design by Torben Mogensen (see <https://link.springer.com/book/10.1007/978-0-85729-829-4>, via TU/e library)
- **Chapter 7 (“Implementation of Textual Concrete Syntax”) of Software Languages by Ralf Lämmel**
- Online material on Rascal: <https://www.rascal-mpl.org/docs/Rascal/Declarations/SyntaxDefinition/> (*and see previous lecture’s slides*)

Context-free syntax

What is syntactical analysis?

- After lexical analysis (scanning), we have a *series of tokens*.
- In *syntax analysis (or parsing)*, we want to interpret what those tokens mean.
- Two goals:
 - *Recover the structure described* by that series of tokens.
 - *Report errors* if those tokens do not properly encode a structure.

Context-free syntax

Limits of regular languages

- When scanning, we used regular expressions to define each token
- Unfortunately, regular expressions are (usually) *too weak to define programming languages*.
 - Cannot define a regular expression matching all expressions with *properly balanced parentheses*
 - Cannot define a regular expression matching all functions with *properly nested block structure*

→ We need a more powerful formalism: context-free grammars (CFGs)

Context-free syntax

CFG definition

A *context-free grammar* is a 4-tuple $G = (N, \Sigma, P, S)$

1. N is a set of *non-terminals*
2. Σ is a set of *terminals* (disjoint from N)
3. P is a subset of $N \times (N \cup \Sigma)^*$
An element $(A, \alpha) \in P$ is called a *production (rule) a.k.a. rule*
 $A ::= \alpha$ (a.k.a. $A \rightarrow \alpha$)
4. $S \in N$ is the *start symbol*

The sets N, Σ, P are finite

Context-free syntax

CFG as rewrite system

A context-free grammar can be considered as a simple **rewrite system** with *rewrite steps*:

if $A ::= \gamma \in P$,

$$\alpha A \beta \Rightarrow \alpha \gamma \beta \quad (\alpha, \beta, \gamma \in (N \cup \Sigma)^*, A \in N)$$

NB: this covers case $S \Rightarrow \dots$

Example $N = \{E\}$, $\Sigma = \{+, *, (,), -, a\}$, $S = E$,

$$P = \left\{ \begin{array}{l} E ::= E + E \\ E ::= E * E \\ E ::= (E) \\ E ::= - E \\ E ::= a \end{array} \right\}$$

Derivation $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(a+E) \Rightarrow -(a+a)$

Context-free syntax

CFG language and sentences

The *language* $L(G)$ generated by the CFG $G = (N, \Sigma, P, S)$ is:

$$L(G) = \{w \in \Sigma^* \mid S \Rightarrow^+ w\}$$

A *sentence* $w \in L(G)$ contains *only terminals*

A *sentential form* α is a string of terminals and non-terminals which can be derived from S :

$$S \Rightarrow^* \alpha \text{ with } \alpha \in (N \cup \Sigma)^*$$

→ A *sentence* in $L(G)$ is a sentential form in which *no* non-terminals occur

Context-free syntax

CFG derivations and yields

A sequence of steps is called a *derivation*.

- A string $\alpha A \omega$ *yields* string $\alpha \gamma \omega$ iff $A \rightarrow \gamma$ is a production (i.e. $A ::= \gamma \in P$).
If α *yields* β , we write $\alpha \Rightarrow \beta$
- We say that α *derives* β iff there is a sequence of strings where
 $\alpha \Rightarrow \alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \beta$
If α *derives* β , we write $\alpha \Rightarrow^* \beta$.

Context-free syntax

Left/right derivations

- There are choices to be made for each derivation step:
 - *which non-terminal* must be replaced?
 - *which alternative of the selected non-terminal* (i.e. *which rule*) must be applied?
- *Always* selecting the leftmost non-terminal in the sentential form gives a *leftmost derivation*: $\Rightarrow_{lm}$
 - There exists also a *rightmost* derivation: $\Rightarrow_{rm}$
- Consider the context-free grammar for expressions:
 - Leftmost derivation for $-(a+a)$: $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(a+E) \Rightarrow -(a+a)$
 - Rightmost derivation for $-(a+a)$: $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(E+a) \Rightarrow -(a+a)$

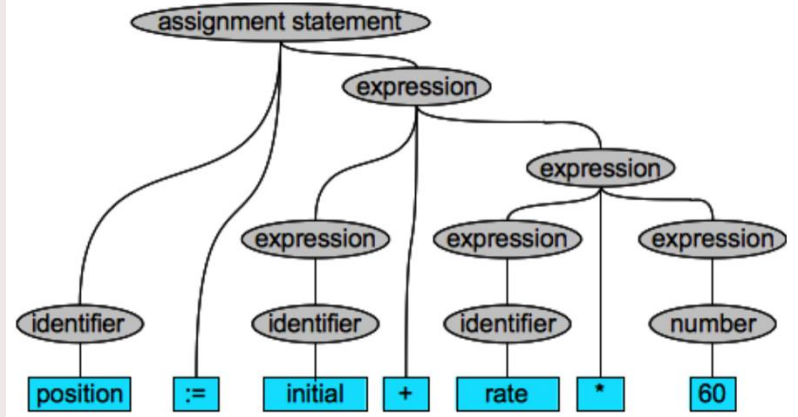
Context-free syntax

Parse trees

- A parse tree is a *tree encoding the steps in a derivation*.
- *Internal nodes* represent *nonterminal symbols* used in the production.
- *In order walk of the leaves* contains the generated string.
- Encodes *what productions* are used, *not the order* in which those productions are applied

A parse tree for the sentence

position := initial + rate * 60

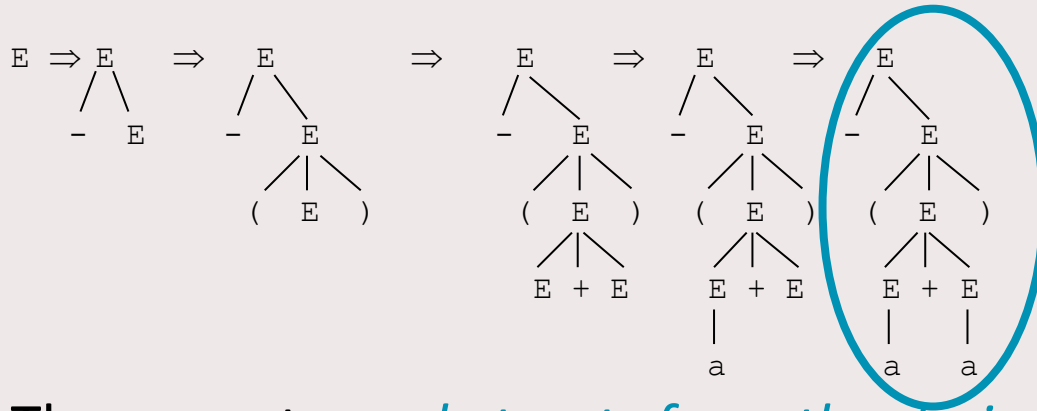


(Source: <https://www.rascal-mpl.org/docs/Rascalopedia/ParseTree/>)

Context-free syntax

Example:

$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(a+E) \Rightarrow -(a+a)$

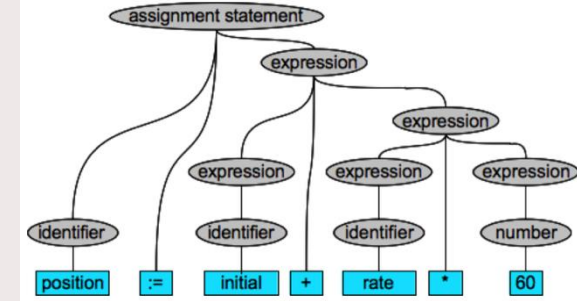


The parse tree *abstracts from the derivation order*

Context-free syntax

A parse tree for a CFG $G = (N, \Sigma, P, S)$ is a tree:

1. The *root* is labeled with S (the start non-terminal)
2. Each *leaf* is labeled with a terminal ($\in \Sigma$) or ε
3. All *other nodes* are labeled with a non-terminal



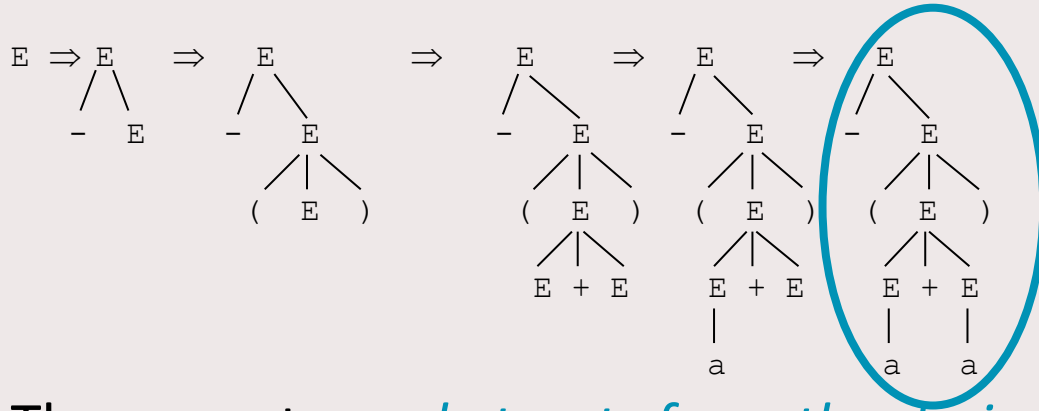
If A is the label of a node and $X_1, \dots, X_n$ the labels of the children (left to right) then $A ::= X_1, \dots, X_n$ **must** be a production rule in G (where each X_i is either a terminal or a non-terminal)

Special case is $A ::= \varepsilon$ with label A which has exactly one child with label ε , also called a *nullable* non-terminal

Context-free syntax

Example:

$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(a+E) \Rightarrow -(a+a)$



The parse tree *abstracts from the derivation order*

Context-free syntax

Acceptor and parser

For each grammar G there exists a decision procedure (*acceptor*) AG for $L(G)$:

$$AG: \Sigma^* \rightarrow \{true, false\}$$

such that

$$AG(w) = true \Leftrightarrow w \in L(G)$$

A *parser* is an acceptor which constructs a parse tree as well.

- A *top-down* parser constructs the tree starting from the root
- A *bottom-up* parser constructs the tree starting from the leaves

Context-free syntax

Goals of parsing

- *Recover the structure* described by a series of tokens.
- If language is described as a CFG, goal is to *recover a parse tree for the input string*.
- If the input string is syntactically incorrect: *generate an error message*

Context-free syntax

During parsing the following problems may occur:

- The grammar is ambiguous
- The grammar is left recursive
- The grammar contains cycles
- Grammars are in principle not modular

Context-free syntax

Syntax definitions in Rascal

- Rascal offers a modular way of defining the syntax of a language

```
module demo::lang::Pico::Syntax  
  
import Prelude  
  
...
```

Context-free syntax

Syntax definitions in Rascal

- The start non-terminal is explicitly defined as follows

```
start syntax Program =  
  program: "begin" Declarations decls {Statement ";"}* body "end";  
...
```

(name of the Abstract Syntax Tree node)

(names of attributes of the AST node)

Context-free syntax

Syntax definitions in Rascal

- Normal production rules:

```
...
syntax Declarations =
    "declare" {Declaration ","}* decls ";" ;

syntax Declaration = decl: Id id ":" Type tp ;

syntax Type =
    natural: "natural"
    | string: "string" ;

...
```

Context-free syntax

Syntax definitions in Rascal

- Normal production rules:

```
...
syntax Statement =
  asgnStat: Id var ":" Expression val
  | ifElseStat: "if" Expression cond "then" {Statement ";" }* thenPart
               "else" {Statement ";" }* elsePart "fi" ;
  | whileStat: "while" Expression cond "do" {Statement ";" }* body "od" ;
...
```

Context-free syntax

Syntax definitions in Rascal

- Normal production rules:

disambiguation via priorities of production rules

```
...
syntax Expression =
  id: Id name
  | strCon: String string
  | natCon: Natural natural
  | bracket: "(" Expression expr ")" ;
  > left conc: Expression lhs "||" Expression rhs
  > left ( add: Expression lhs "+" Expression rhs
    | sub: Expression lhs "-" Expression rhs
  ) ;
...
```

disambiguation via left associativity of binary operators

Questions?



Context-free syntax

During parsing the following problems may occur:

- The grammar is ambiguous
- The grammar is left recursive
- The grammar contains cycles
- Grammars are in principle not modular

When is a grammar ambiguous?

A grammar G is *ambiguous* if some word $w \in L(G)$ has **at least two parse trees**

- Expression grammar without associativities and priorities
- Dangling else problem

Context-free syntax

Ambiguity

- A CFG is said to be ambiguous if there is at least one string with two or more parse trees.
- Note that *ambiguity is a property of a grammar*, not necessarily of the corresponding language.
- There is *no algorithm for converting an arbitrary ambiguous grammar into an unambiguous one*.
 - Some languages are inherently ambiguous, meaning that no unambiguous grammar exists for them.
- There is *no algorithm for detecting whether an arbitrary grammar is ambiguous*.

Context-free syntax

Is ambiguity a problem?

- depends on semantics

$E ::= \text{int} \mid E + E$

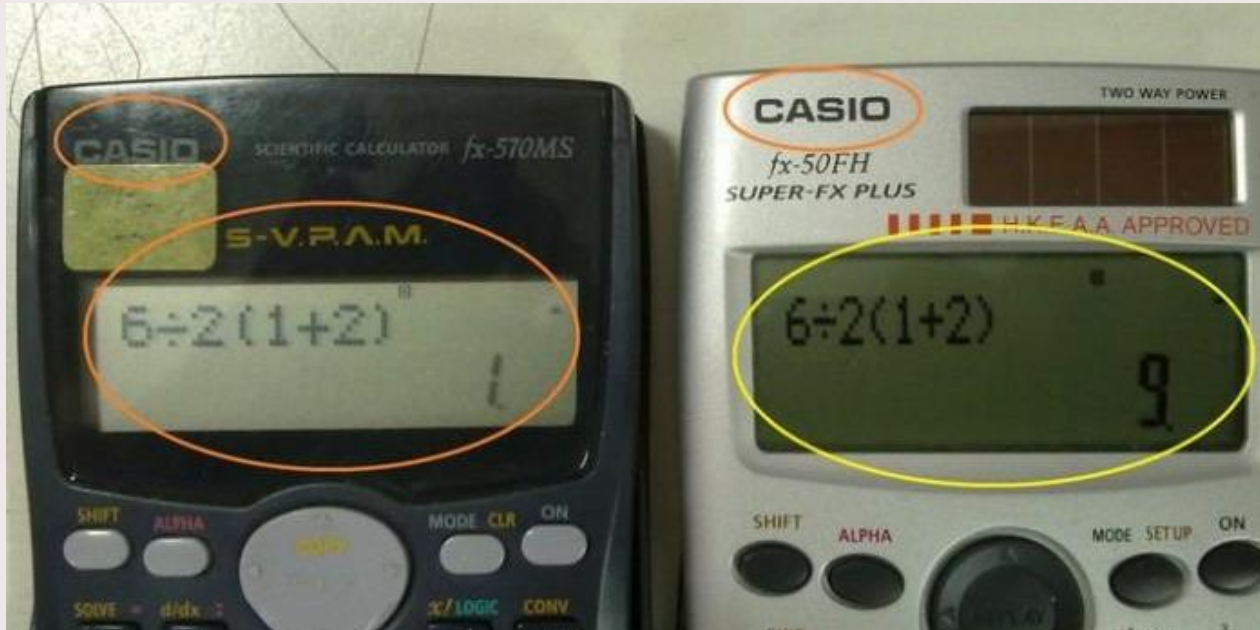
$1 + 2 + 3$

VS

$E ::= \text{int} \mid E + E \mid E - E$

$5 - 3 + 7$

Context-free syntax



Context-free syntax

Resolving ambiguities

- If a grammar can be made unambiguous at all, it is usually made unambiguous *through layering*.
 - Have exactly *one way to build each piece* of the string.
 - Have exactly *one way of combining those pieces* back together.

Context-free syntax

Example of balanced parentheses

- Consider the language of all strings of balanced parentheses:
 - ε
 - $()$
 - $(())()$
 - $((()))(())()$
- A possible grammar for balanced parentheses is:
$$S ::= \varepsilon \mid S S \mid (S)$$
 - Why is this grammar ambiguous?
 - How to resolve this ambiguity?

Context-free syntax

Restructuring the grammar of parentheses

- A string of balanced parentheses is a sequence of strings that are themselves balanced parentheses
- To avoid ambiguity, we can **build the string in two steps**:
 - **Decide how many different substrings** we will glue together.
 - **Build each substring** independently.

Context-free syntax

Building a new grammar for parentheses

- Spread a string of parentheses across the string. There is exactly one way to do this for any number of parentheses.
- Expand out each substring by adding in parentheses and repeating.

Another grammar for balanced parentheses is:

$$\begin{aligned} S &::= P S \mid \varepsilon \\ P &::= (S) \end{aligned}$$

Context-free syntax

Why **explicit disambiguation** in Rascal?

- **Fully declarative** definition of syntax
- Implicit disambiguation can give unexpected results
- More programming languages can be parsed:
 - legacy languages
- Modularity
- **Fewer non-terminals/sorts**
- **Separation of concerns**: form of rules is independent of disambiguation

Questions?



Context-free syntax

During parsing the following problems may occur:

- The grammar is ambiguous
- The grammar is left recursive
- The grammar contains cycles
- Grammars are in principle not modular

Context-free syntax

Left recursion removal:

- A grammar is *immediately left recursive* if the grammar contains a rule of the form $A ::= A\alpha$
- A grammar is *left recursive* if there exists a non-terminal A and a string $\alpha \in (N \cup \Sigma)^*$ such that $A \Rightarrow^* A\alpha$
- This means that after *one or more steps in a derivation an occurrence of A reduces again to an occurrence of A without recognizing any terminal* in the input sentence.

Context-free syntax

Examples of *indirect* left recursion

$$A ::= B \alpha$$

$$B ::= A \beta$$

or worse

$$A ::= B \alpha$$

$$B ::= D A \beta$$

$$D ::= \varepsilon \mid \gamma G$$

It is easy to remove left recursion from a context-free grammar

Elimination rule for **immediate** left recursion

$$A ::= A \alpha$$
$$A ::= \beta$$

where $\alpha \neq \varepsilon$

produces $\beta \alpha^n$

A set of equivalent (non left recursive) rules are

$$A ::= \beta A'$$
$$A' ::= \alpha A'$$
$$A' ::= \varepsilon$$

Context-free syntax

Example:

- (1) $E ::= E + T$ (immediate left rec.)
- (2) $\quad \quad | T$
- (3) $T ::= T * F$ (immediate left rec.)
- (4) $\quad \quad | F$
- (5) $F ::= (E)$
- (6) $\quad \quad | a$

Applying the left recursion elimination transformation:

- (1) $E ::= E \alpha$ (with $\alpha = + T$, and $+ T \neq \epsilon$)
- (2) $E ::= \beta$ (with $\beta = T$)

Context-free syntax

Example:

(1') $E ::= T E'$

(2') $E' ::= + T E'$

(2'') $\mid \varepsilon$

the same for:

(3') $T ::= F T'$

(4') $T' ::= * F T'$

(4'') $\mid \varepsilon$

Elimination rule for **indirect** left recursion

- Suppose we have a rule of the form

$$\begin{array}{l} A ::= B \alpha \\ B ::= \beta_1 \\ \quad | \beta_2 \\ \quad \vdots \\ \quad | \beta_n \end{array}$$

- The rule $A ::= B \alpha$ is now transformed into:

$$\begin{array}{l} A ::= \beta_1 \alpha \\ \quad | \beta_2 \alpha \\ \quad \vdots \\ \quad | \beta_n \alpha \end{array}$$

Elimination process for indirect left recursion

This process is repeated until either

- $A ::= t \gamma$; the process stops, or
- $A ::= A \gamma$; the *immediate left recursion elimination rule* can be applied

Left factorization

- In general it is efficient to move the difference between the alternatives of a non-terminal as far as possible to the left

- Productions of the form

$$\begin{aligned} A &::= \alpha \beta_1 \\ &\quad | \alpha \beta_2 \\ &\quad \dots \\ &\quad | \alpha \beta_n \end{aligned}$$

- Are equivalent to

$$\begin{aligned} A &::= \alpha A' \\ A' &::= \beta_1 \\ &\quad | \beta_2 \\ &\quad \dots \\ &\quad | \beta_n \end{aligned}$$

Context-free syntax

Example

$S ::= \text{if } B \text{ then } S \text{ else } S$
 $\quad \quad \quad | \text{if } B \text{ then } S$

- Only at the occurrence of `else` it can be decided which alternative should have been selected
- An equivalent grammar is

$S ::= \text{if } B \text{ then } S S'$
 $S' ::= \text{else } S$
 $\quad \quad \quad | \varepsilon$

Context-free syntax

Associativity and priorities:

```
Bool      : Bool '|' Bool;  
Bool      : Bool '&' Bool;  
Bool      : 'not' Bool;  
Bool      : '(' Bool ')';  
Bool      : BoolCon;  
BoolCon   : 'true' | 'false';
```

Adding (left) associativity and priorities by means of *layering*:

```
OrBool     : OrBool '|' AndBool;  
OrBool     : AndBool;  
AndBool    : AndBool '&' NotBool;  
AndBool    : NotBool;  
NotBool    : 'not' BracketBool;  
NotBool    : BracketBool;  
BracketBool : '(' OrBool ')';  
BracketBool : BoolCon;
```

Context-free syntax in Rascal

Left recursion removal:

```
OrBool    : AndBool OrBool'  
OrBool'   : '|' AndBool OrBool'  
          |  $\epsilon$ ;  
AndBool   : NotBool AndBool';  
AndBool'  : '&' NotBool AndBool'  
          |  $\epsilon$  ;  
NotBool   : 'not' BracketBool;  
NotBool   : BracketBool;  
BracketBool : '(' OrBool ')';  
BracketBool : BoolCon;
```


Questions?

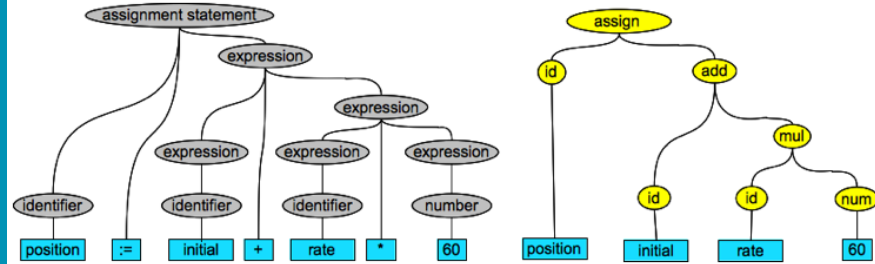


Abstract syntax

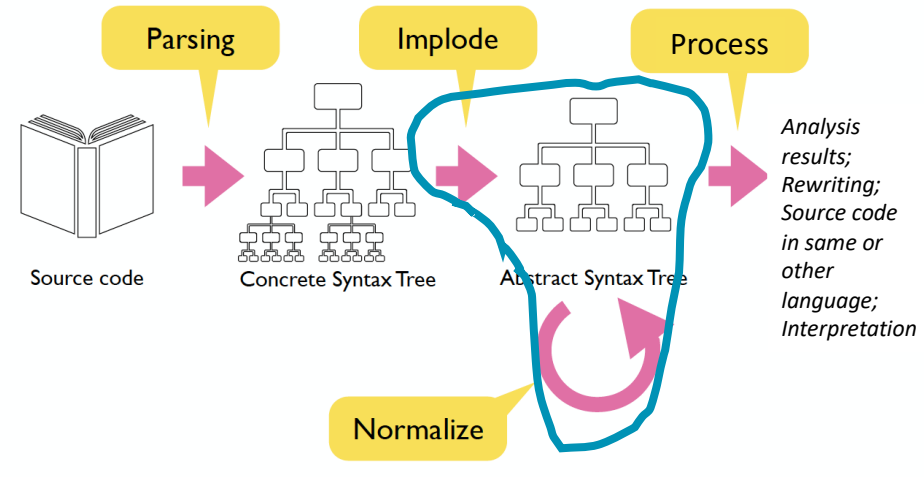
For the input sentence

position := initial + rate * 60

the parse tree (left) and abstract syntax tree (right) may look as follows:



(Source: <https://www.rascal-mpl.org/docs/Rascalopedia/ParseTree/>)

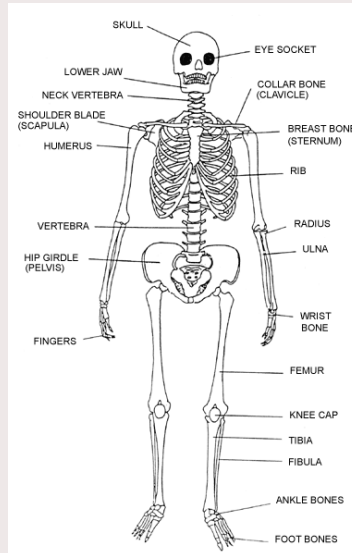


Abstract syntax

Abstract syntax defines basic structure of the language (skeleton) and is* starting point for defining:

- concrete syntax
- static semantics
- dynamic semantics

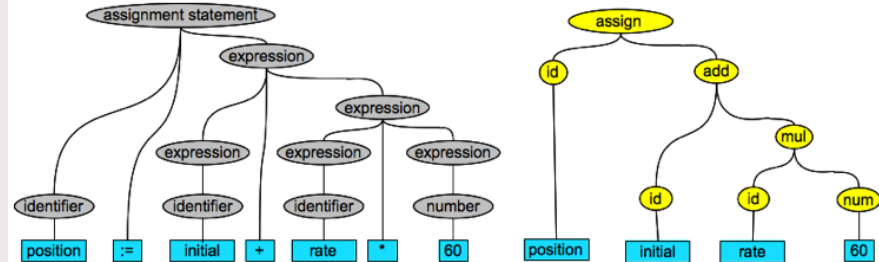
* But in grammarware setting...?



For the input sentence

position := initial + rate * 60

the parse tree (left) and abstract syntax tree (right) may look as follows:



(Source: <https://www.rascal-mpl.org/docs/Rascalopedia/ParseTree/>)

Recall: Context-free syntax

Syntax definitions in Rascal

- The start non-terminal is explicitly defined as follows

```
start syntax Program =  
  program: "begin" Declarations decls {Statement ";" }* body "end";  
...
```

name of the abstract syntax
tree node

name of the attributes of the AST
node

Abstract syntax

- Abstract syntax is a **collection of constructors/-functions**
- No information about keywords, priorities, associativities, etc.

Abstract syntax

Signature describes it as follows:

- A collection of *functions that define sorts and operators*
- A *sort* (*non-terminal, type*) represents a nonempty set of *terms*
- A *k-ary operator* is a *constructor function* mapping *k* terms of the appropriate *sorts* to a term of the result *sort*
- Format of the functions:

$$sort_0 = op(sort_1, sort_2, \dots, sort_k)$$

Abstract syntax

Mapping from concrete syntax rule into abstract syntax rule (in Rascal):

```
syntax Expression = id: Id name
                    | natCon: Natural natcon
                    ...
                    | add: Expression left "+" Expression right;

public data EXP = id(Id name)
                  | natCon(int iVal)
                  ...
                  | add(EXP left, EXP right);
```

name of AST node

name of attribute of the AST node

Concrete

vs.

Abstract

Abstract syntax

Abstract syntax of Pico toy language in Rascal

```
public data PROGRAM = program(list[DECL] decls, list[STATEMENT] stats);  
public data DECL = decl(Id name, TYPE tp);  
public data EXP = id(Id name)  
    | natCon(int iVal)  
    | strCon(str sVal)  
    | add(EXP left, EXP right)  
    | sub(EXP left, EXP right)  
    | conc(EXP left, EXP right);  
  
public data STATEMENT =  
    asgStat(Id name, EXP exp)  
    | ifElseStat(EXP exp, list[STATEMENT] thenpart, list[STATEMENT] elsepart)  
    | whileStat(EXP exp, list[STATEMENT] body);
```


Questions?



